

CO2 - ASSESSMENT TOOL 1
ANALYTICAL PROBLEM SOLVING

NAME: S. GAYATHRI
REG NO: 192521306
COURSE CODE: CSA06
COURSE NAME: Design and Analysis of Algorithms

88%

Q12: Bubble Sort - Duplicate Values Analysis

Apply the Brute Force technique using Bubble Sort on the dataset $[32, 14, 32, 7, 19, 7]$. Clearly Interpret the problem and explain why Bubble Sort can be applied to datasets with duplicate values. Perform the sorting process step-by-step and display intermediate results after each pass. Count the Number of Comparisons and Swaps performed. Analyze the time complexity in best, average and worst cases. Verify the correctness of the final sorted output and Interpret the behavior of the algorithm on repeated elements.

Solution

PROBLEM INTERPRETATION

The Given dataset is

$[32, 14, 32, 7, 19, 7]$

The Task is to apply the Bubble Sort algorithm to sort the Array in ascending order. The dataset contains duplicate values (32 & 7). Bubble Sort repeatedly compares adjacent elements E_i and E_{i+1} . Swaps them if they are in the wrong order until the array becomes sorted.

Why Bubble Sort is Suitable for Duplicate Values?

$\Rightarrow$ It is a Comparison-Based Brute Force

Sorting algorithm

$\Rightarrow$ It is a stable sorting algorithm, meaning duplicate elements maintain their original relative order.

$\Rightarrow$ It compares only adjacent elements

$\Rightarrow$ It correctly sorts arrays containing duplicate values

$\Rightarrow$ It is simple to understand and implement

STEP-by-STEP BUBBLE SORT

Initial Array = $[32, 14, 32, 7, 19, 7]$

PASS 1

$32 > 14 \rightarrow \text{Swap}$

$32 = 32 \rightarrow \text{No Swap}$

$32 > 7 \rightarrow \text{Swap}$

$32 > 19 \rightarrow \text{Swap}$

$32 > 7 \rightarrow \text{Swap}$

Array after PASS 1 : $[14, 32, 7, 19, 7, 32]$

Comparisons = 5

Swaps = 4

PASS 2

$14 \neq 32 \rightarrow \text{No Swap}$

$32 > 7 \rightarrow \text{Swap}$

$32 > 19 \rightarrow \text{Swap}$

$32 > 7 \rightarrow \text{Swap}$

Array after pass 2 : $[14, 7, 19, 7, 32, 32]$

Comparisons = 4

Swaps = 3

PASS 3

$14 > 7 \rightarrow \text{Swap}$

$14 < 19 \rightarrow \text{No Swap}$

$19 > 7 \rightarrow \text{Swap}$

Array after PASS 3 : $[7, 14, 7, 19, 32, 32]$

Comparisons = 3

Swaps = 2

PASS 4

$7 < 14 \rightarrow \text{No Swap}$

$14 > 7 \rightarrow \text{Swap}$

Array after PASS 4 : $[7, 7, 14, 19, 32, 32]$

Comparisons = 2

Swaps = 1

PASS 5

$7 = 7 \rightarrow$ No Swap

Array after PASS 5 : $[7, 7, 14, 19, 32, 32]$

Comparisons = 1

Swaps = 0

Final Sorted OUTPUT : $[7, 7, 14, 19, 32, 32]$ Ascending order

TOTAL COMPARISONS : 15

TOTAL SWAPS : 10

TIME Complexity Analysis

BEST CASE

$O(n)$ - If the Array is already sorted, only one pass is needed (with optimization using a swap flag)

AVERAGE CASE

$O(n^2)$ - Random Input requires multiple passes & comparisons

WORST CASE

$O(n^2)$

Reverse - sorted arrays require the Maximum Number of Comparisons and Swaps

SPACE

Complexity

$\Rightarrow O(1)$ [Constant space]

Correctness Verification

Final Sorted Array : $[7, 7, 14, 19, 32, 32]$

Verification : $7 \leq 7 \leq 14 \leq 19 \leq 32 \leq 32$

Hence, the array is correctly sorted in Ascending order

Behavior on Repeated Elements

- $\Rightarrow$ Bubble Sort correctly handles duplicate values without removing them
- $\Rightarrow$ Duplicate elements (7 & 32) appear together in a final sorted array
- $\Rightarrow$ Since Bubble Sort is stable, equal elements maintain their original relative order.
- $\Rightarrow$ The Algorithm continues making comparisons with duplicate values but performs no unnecessary swaps when adjacent elements are equal

CONCLUSION

Bubble Sort successfully sorted the dataset into $[7, 7, 14, 19, 32, 32]$ while correctly handling duplicate values. It performed 15 comparisons & 10 swaps. Although simple & stable, its $O(n^2)$ time complexity makes it suitable mainly for small datasets. Therefore Bubble Sort is easy to understand & reliable for sorting small datasets containing repeated elements.